

github commits: <https://github.com/hnguy130/CSE-464-2025-hnguy130/commits/refactor/>

The screenshot shows the GitHub Commits page for the 'refactor' branch. At the top, there's a filter for 'refactor' and buttons for 'All users' and 'All time'. Below this, the commits are grouped by date. The first group is for 'Apr 29, 2025' and contains one commit: 'Adding readme.pdf for PROJECT 2' by hnguy130, committed 35 minutes ago, with hash 4857f19. The second group is for 'Apr 25, 2025' and contains five commits: 'Example graph and example search' (hash 6a38da1, committed 3 days ago), 'Task 4' (hash 91b35d1, committed 4 days ago), 'Task 3' (hash 213835f, committed 4 days ago), 'Task 2' (hash 300625d, committed 4 days ago), and 'Task 1. 5 refactoring' (hash 73e256c, committed 4 days ago). Each commit entry includes the commit hash, a copy icon, and a compare icon.

github workflow

The screenshot shows the GitHub Actions workflow page for 'Project 3 demo #39'. The workflow is named 'Java CI with Maven' and is currently in a 'Success' state. It was triggered by a push to the 'main' branch by hnguy130. The total duration of the workflow is 21s. The workflow file is 'maven.yml' and it runs on 'push'. The workflow consists of one job named 'build' which took 15s to complete. The page also shows a 'Summary' tab and a 'Re-run all jobs' button.

github merge (merge didn't show as merge but as commits)

This screenshot is a zoomed-in view of the 'Apr 25, 2025' commit group from the previous screenshot. It shows the following commits in order: 'Example graph and example search' (hash 6a38da1, committed 3 days ago), 'Task 4' (hash 91b35d1, committed 4 days ago), 'Task 3' (hash 213835f, committed 4 days ago), 'Task 2' (hash 300625d, committed 4 days ago), and 'Task 1. 5 refactoring' (hash 73e256c, committed 4 days ago). Each commit entry includes the commit hash, a copy icon, and a compare icon.

1.5 refactoring

github link:

<https://github.com/hnguy130/CSE-464-2025->

[hnguy130/commit/73e256c10a17bcdcab2b0358e98ab340ce3206b5](https://github.com/hnguy130/commit/73e256c10a17bcdcab2b0358e98ab340ce3206b5)

```
//refactor 1
String ERROR_NO_GRAPH = "No graph loaded into system\n";
String ERROR_NO_FILE = "Graph txt not found: ";

//refactor 2
public boolean graphExists() {
    if (graph == null) {
        System.out.println(ERROR_NO_GRAPH);
        return false;
    }
    else
        return true;
}

//refactor 3
public File fileExists(String filepath) throws IOException {
    File txt = new File(filepath);
    if (!txt.exists()) {
        throw new IOException(ERROR_NO_FILE + filepath);
    }
    return txt;
}

//refactor 4
public MutableNode nodeExists(String label) {
    for (MutableNode node : graph.nodes()) {
        if (node.name().value().equals(label))
            return node;
    }
    return null;
}

//refactor 5
public ArrayList<String> buildPath(String end, String start, HashMap<String, String> parent){
    ArrayList<String> path = new ArrayList<>();
    String node = end;

    while (!node.equals("none")) {
        path.add(0, node);
        node = parent.get(node);
    }

    return path;
}
```

- refactoring 1: ERROR_NO_GRAPH, ERROR_NO_FILE

Type: Replace Magic Number with Symbolic Constant

Reason: To reduce typing the same string over again and avoid slight mistakes

- refactoring 2: graphExists()

Type: Extract method

Reason: To check if graph exists and print error if graph doesn't exist, return Boolean for further processing

- refactoring 3: fileExists(String filepath)

Type: Extract method

Reason: To check if file exists in file system and print error if file doesn't exist, return Boolean for further processing

- refactoring 4: nodeExists(String label)

Type: Extract method

Reason: To check through Graph's list of nodes if a node is in that list of nodes, and return that node or null if node doesn't exist

- refactoring 5: buildPath(String end, String start, HashMap<String, String> parent)

Type: Extract method

Reason: To create an arraylist that contains path of nodes from child node to start node, and avoid redundancy and complexity

2.

github link:

<https://github.com/hnguy130/CSE-464-2025-hnguy130/commit/300625d5dd7d059bf07c241df156c01e55e3d57d>

- The class templateSearch is the template for subclasses that implement search types Breadth-first-search, Depth-first-search

+ Class templateSearch's skeleton for algorithm processes is the method search(String from, String to). Method search(String from, String to) includes the common steps of the algorithms BFS, DFS

+ checking if there is a graph

+ checking if the start and the destination do exist in the graph, if not, return

+ Then, the skeleton method calls the abstract method searchHelper(String from, String to). This abstract method searchHelper(String from, String to) has to be implemented once for each subclass, with each subclass only able to contain one search algorithm, totaling 2 subclasses for BFS, DFS. The subclasses are: searchBFS and searchDFS

```
//template class for pattern template
public abstract class templateSearch {

    public Path search(String from, String to) {
        if(graphExists() == false)
            return null;

        if(nodeExists(from) == null)
            throw new IllegalArgumentException("Node " + from + " as source node doesn't exist in graph\n");

        if(nodeExists(to) == null)
            throw new IllegalArgumentException("Node " + to + " as a node doesn't exist in graph\n");

        Path path = searchHelper(from, to);
        return path;
    }

    public abstract Path searchHelper(String from, String to);
}
```

- Each subclass searchBFS, searchDFS, has the skeleton method search(String from, String to), but they need to implement their own searchHelper(String from, String to) separately for each subclass
 - + searchHelper(String from, String to) in searchBFS subclass implements the search using Breath-first-search algorithm
 - + searchHelper(String from, String to) in searchDFS subclass implements the search using Depth-first-search algorithm
- After implementing the different parts for each subclass about one search algorithm in DFS, BFS, their common skeleton method search(String from, String to):
 - + processes all the common steps of all algorithms,
 - + processes the specific algorithm's separate parts in the abstract method searchHelper(String from, String to) differently

```
public class searchBFS extends templateSearch {
    @Override
    public Path searchHelper(String from, String to) {
        ArrayList<String> queue = new ArrayList<>();
        ArrayList<String> visited = new ArrayList<>();
        HashMap<String, String> parent = new HashMap<>();

        queue.add(from);
        visited.add(from);
        parent.put(from, "none");

        while (!queue.isEmpty()) {
            String current = queue.remove(0);

            for (Link link : edgeList) {
                if (link.from().name().value().equals(current)) {
                    String child = link.to().name().value();

                    if (!visited.contains(child)) {
                        queue.add(child);
                        visited.add(child);
                        parent.put(child, current);

                        if (child.equals(to)) {
                            ArrayList<String> path = buildPath(child, from, parent);
                            return new Path(path, "BFS");
                        }
                    }
                }
            }
        }

        return null;
    }
}
```

```
public class searchDFS extends templateSearch {
    @Override
    public Path searchHelper(String from, String to) {
        ArrayList<String> stack = new ArrayList<>();
        ArrayList<String> visited = new ArrayList<>();
        HashMap<String, String> parent = new HashMap<>();

        stack.add(from);
        visited.add(from);
        parent.put(from, "none");

        while (!stack.isEmpty()) {
            String current = stack.get(stack.size()-1);

            for (Link link : edgeList) {
                for (Link edge : edgeList) {
                    if (edge.from().name().value().equals(current)) {
                        String next = edge.to().name().value();

                        if (!visited.contains(next)) {
                            stack.add(next);
                            visited.add(next);
                            parent.put(next, current);
                            current = next;

                            if (current.equals(to)) {
                                ArrayList<String> path = buildPath(current, from, parent);
                                return new Path(path, "DFS");
                            }
                        }
                    }
                }
            }

            stack.remove(stack.size()-1);
        }

        return null;
    }
}
```

3.

github link:

<https://github.com/hnguy130/CSE-464-2025->

[hnguy130/commit/213835f7f347262aa4a0f242c467a566e889d4d6](https://github.com/hnguy130/commit/213835f7f347262aa4a0f242c467a566e889d4d6)

```
public class strategyBFS implements searchStrategy{
    public Path search(String from, String to) {
        searchBFS bfs = new searchBFS();
        return bfs.search(from, to);
    }
}

public class strategyDFS implements searchStrategy{
    public Path search(String from, String to) {
        searchDFS dfs = new searchDFS();
        return dfs.search(from, to);
    }
}

public class strategyRANDOM implements searchStrategy{
    public Path search(String from, String to) {
        searchRANDOM random = new searchRANDOM();
        return random.search(from, to);
    }
}

public class Context {
    public searchStrategy searchType;

    public Context(Algorithm algo) {
        if(algo == Algorithm.BFS) {
            searchType = new strategyBFS();
        }
        else if (algo == Algorithm.DFS) {
            searchType = new strategyDFS();
        }
        else if (algo == Algorithm.RANDOM) {
            searchType = new strategyRANDOM();
        }
        else
            searchType = null;
    }

    public Path search(String from, String to) {
        if(searchType != null)
            return searchType.search(from, to);
        else
            return null;
    }
}

public Path GraphSearch(String from, String to, Algorithm algo){

    Context searcher = new Context(algo);

    System.out.println("-----\n");

    if(algo == Algorithm.BFS)
        System.out.println(" Breath first search: \n");
    else if (algo == Algorithm.DFS)
        System.out.println(" Depth first search: \n");
    else if (algo == Algorithm.RANDOM)
        System.out.println(" Random walk search: \n");

    Path path = searcher.search(from, to);
    System.out.println(path.toString());

    System.out.println("-----");

    return path;
}
```

Reference:

- method GraphSearch(String from, String to, Algorithm algo)
- > Context class
- > searchStrategy interface (strategy interface)
- > strategyBFS or strategyDFS class (classes for strategy)
- > searchBFS or searchDFS class (classes for algorithm)

- The class Context manages the input and selects the correct strategy to process. The structure of the class Context allows selecting the correct search algorithm BFS or DFS:

- + Context class has the function search(String from, String to) which returns a path

- + A Context object is created, with a parameter for the specific algorithm, using
Context searcher = new Context(algo);

- + (point 1) After knowing the specific algorithm, the Context object creates the specific concrete strategy class' object strategyBFS or strategyDFS, from interface searchStrategy:

- + (point 2) Then, strategyBFS or strategyDFS object's custom function search(String from, String to) returns the specific algorithm class searchBFS or searchDFS's abstract function search(String from, String to)

- + To search a path, use searcher.search(String from, String to). T

4.

github link:

<https://github.com/hnguy130/CSE-464-2025-hnguy130/commit/91b35d10055db1fcef2d618f726d20953e1345d3>

- You must implement this new random walk algorithm using both the strategy pattern and the template pattern you implement for requirements 2 and 3 in this part.

+ Updated template pattern implementation

```
public class searchRANDOM extends templateSearch {
    Random random = new Random();

    @Override
    public Path searchHelper(String from, String to) {
        // try finding path for at most 100 searches, then return null

        ArrayList<String> path;
        ArrayList<String> visited;
        ArrayList<String> adjacent;

        System.out.println("RANDOM: \n");

        for(int i = 0; i < 100; i++) {
            path = new ArrayList<>();
            path.add(from);
            visited = new ArrayList<>();
            visited.add(from);
            adjacent = new ArrayList<>();

            //current node at start
            String current = from;

            System.out.println(" - New Search - ");

            //check if there are adjacent nodes
            // if none, return null path
            //if yes, continue searching
            for(Link edge : edgeList) {
                if(edge.from().name().value().equals(current))
                    adjacent.add(edge.to().name().value());
            }

            if(adjacent.isEmpty())
                return null;

            boolean dead_end = false;

            while (dead_end == false) {
                //selecting random adjacent node
                boolean all_visited = true;
                for(String node : adjacent)
                    if(visited.contains(node) == false) {
                        all_visited = false;
                        break;
                    }

                if(all_visited == true) {
                    printPath(path);
                    System.out.println(" - Dead End - \n");
                    break;
                }

                current = adjacent.get(random.nextInt(adjacent.size()));
                while(visited.contains(current)) {
                    current = adjacent.get(random.nextInt(adjacent.size()));
                }

                path.add(current);
                visited.add(current);

                if(current.equals(to)) {
                    printPath(path);
                    System.out.println(" - Found Path - ");
                    System.out.println();
                    return new Path(path, "RANDOM");
                }

                adjacent.clear();
                //gathering adjacent nodes
                for(Link edge : edgeList) {
                    if(edge.from().name().value().equals(current))
                        adjacent.add(edge.to().name().value());
                }

                if(adjacent.isEmpty()) {
                    printPath(path);
                    System.out.println(" - Dead End - \n");
                    dead_end = true;
                }
            }
        }

        return null;
    }
}
```


+ Updated strategy pattern implementation

```
public class strategyRANDOM implements searchStrategy{
    public searchRANDOM random = new searchRANDOM();

    public Path search(String from, String to) {
        return random.search(from, to);
    }
}
```

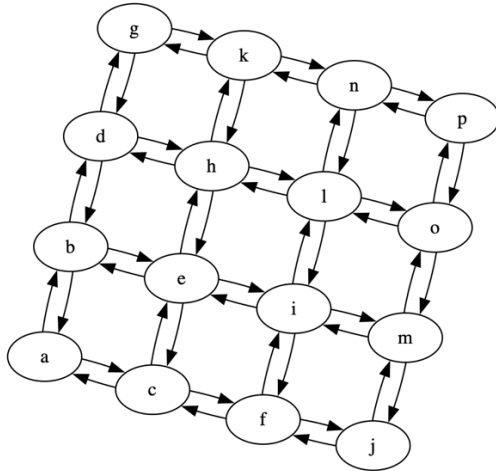
```
public class Context {
    public searchStrategy searchType;

    public Context(Algorithm algo) {
        if(algo == Algorithm.BFS)
            searchType = new strategyBFS();
        else if (algo == Algorithm.DFS)
            searchType = new strategyDFS();
        else if (algo == Algorithm.RANDOM)
            searchType = new strategyRANDOM();
        else
            searchType = null;
    }

    public Path search(String from, String to) {
        if(searchType != null)
            return searchType.search(from, to);
        else
            return null;
    }
}
```

- You must provide an example graph and an example search to find a path in the example graph.

Sample graph:



Example search

```
String start = "a";
String end = "f";
grapher.GraphSearch("a", "f", Algorithm.RANDOM);
```

```
Console X
<terminated> Grapher [Java Application] /Library/Java/JavaVirtualMachines/temurin-21.jdk/Contents/Home/bin/java (Apr 25, 2025, 6:48:06 PM - 6:48:06 PM) [pid: 47051]
vrs: a->u->g->k->n->e->e->i
RANDOM:
- New Search -
visiting Path size 11 - a -> c -> e -> b -> d -> h -> k -> n -> l -> o -> p
- Dead End -

- New Search -
visiting Path size 10 - a -> c -> e -> b -> d -> h -> l -> n -> k -> g
- Dead End -

- New Search -
visiting Path size 13 - a -> b -> e -> h -> d -> g -> k -> n -> p -> o -> l -> i -> f
- Found Path -

RANDOM: a->b->e->h->d->g->k->n->p->o->l->i->f
Graph information written to: graphOutput.txt
```

- You must make your code can search a path from a to c using the provided graph file at canvas

```
Console X
<terminated> Grapher [Java Application] /Library/Java/JavaVirtualMachines/temurin-21.jdk/Contents/Home/bin/java (Apr 25, 2025, 6:53:31 PM - 6:53:31 PM) [pid: 47153]

RANDOM:

- New Search -
visiting Path size 4 - a -> e -> g -> h
- Dead End -

- New Search -
visiting Path size 4 - a -> b -> c -> d
- Dead End -

- New Search -
visiting Path size 3 - a -> e -> f
- Found Path -

RANDOM: a->e->f
```

- You must run the search multiple times and output the search process to show the randomness of your search.

```
RANDOM:

- New Search -
visiting Path size 4 - a -> e -> g -> h
- Dead End -

- New Search -
visiting Path size 4 - a -> b -> c -> d
- Dead End -

- New Search -
visiting Path size 4 - a -> e -> g -> h
- Dead End -

- New Search -
visiting Path size 4 - a -> b -> c -> d
- Dead End -

- New Search -
visiting Path size 3 - a -> e -> f
- Found Path -

RANDOM: a->e->f

<terminated> Grapher [Java Application] /Library/Java/JavaVirtualMachines/temurin-21.jdk/Contents/Home/bin/java (Apr 25, 2025, 6:55:46 PM - 6:55:46 PM) [pid: 47214]
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Graph file to be parsed with: input.dot

Graph information written to: graphData.txt

RANDOM:

- New Search -
visiting Path size 4 - a -> b -> c -> d
- Dead End -

- New Search -
visiting Path size 4 - a -> e -> g -> h
- Dead End -

- New Search -
visiting Path size 4 - a -> b -> c -> d
- Dead End -

- New Search -
visiting Path size 3 - a -> e -> f
- Found Path -

RANDOM: a->e->f
```